Tutorial 2: NVE molecular dynamics of a protein

CIS-1 · Thursday 12 Nov 2026, 2:00–3:00 PM · A3 301

Before class, finish `1_BEFORE_CLASS_Install.pdf`: `python check_install.py` should print `ALL GOOD`. All commands work the same on **Ubuntu and macOS** unless marked otherwise. Just the commands, without explanations, are in `STEPS_TO_RUN.pdf`.

What you will do

You will simulate a small protein in water with two different MD programs, **NAMD** and **OpenMM**, and check a basic law of physics on the result: with no thermostat, **total energy stays constant**.

By the end you should be able to:

1. Read the input and output files of NAMD and OpenMM.
2. Run a simulation in the **NVE ensemble** (constant number of atoms N, volume V, energy E).
3. Show from your own plot that total energy is conserved, that kinetic and potential energy trade off, and that temperature fluctuates by itself.
4. Explain how the timestep affects energy conservation.

The system. Trp-cage (PDB 1L2Y), a 20-residue mini-protein, in a box of TIP3P water: 6,215 atoms in total, Amber ff14SB force field.

Background: what NVE means

- In NVE the atoms follow **Newton's equations and nothing else**. Nothing adds or removes energy, so **E_total = KE + PE is constant**.
- A **thermostat** (fixes temperature) or a **barostat** (fixes pressure) acts like an outside bath that exchanges energy with the system. NVE uses neither.
- We start by giving the atoms random velocities for 300 K. After that nothing controls the temperature: it moves around by itself.
- Why check energy conservation? The equations conserve energy exactly, but a computer solves them in finite steps. Any slow change (**drift**) in E_total is therefore numerical error, from a timestep that is too large or from loose cutoff/PME settings. It is a quick health check of a simulation.

> **Why the start file is "pre-equilibrated".** An NVE run cannot change the box size. A freshly built water box has the wrong density, so it was first relaxed once with a thermostat and barostat (not part of this tutorial). `inputs/trpcage_equil.rst7` is the result. Everything you run today is pure NVE.

What is in the folder

Path	What it is
`inputs/trpcage_wat.prmtop`	topology + force field (used by both engines)
`inputs/trpcage_equil.rst7`	starting coordinates + box (used by both engines)
`namd/nve.namd`	NAMD settings file. Output goes to `namd/out/`
`openmm/nve.py`	OpenMM script
`analysis/plot_nve.py`	plots the energies from both engines
`reference_results/`	complete 60 ps runs of both engines, to compare with yours
`inputs/system_prep/`	how the system was built from the PDB file (you can ignore it)

Step 0. Open the folder

```
cd ~/Downloads/CIS1_Tutorial2_NVE          # or wherever you unzipped it
conda activate cis1-md
python check_install.py                    # expect: ALL GOOD
```

If `conda activate` says `EnvironmentNameNotFound`, or `check_install.py` says `NAMD not on PATH`, you skipped part of the install. The fix commands are in the table at the end of this document.

Look at the starting coordinates file:

```
ls inputs
head -3 inputs/trpcage_equil.rst7          # title line, atom count, then x y z
```

Step 1. Run NAMD

Start this first: it is the slowest step (about 5–15 minutes on a laptop).

```
cd namd
namd2 +p4 nve.namd > out/nve.log
```

- Type `namd3` instead of `namd2` if that is the version you installed.
- `+p4` means "use 4 CPU cores". Use the number of cores your laptop has: `nproc` (Ubuntu) or `sysctl -n hw.ncpu` (macOS).
- Nothing appears on screen: all output goes into `out/nve.log`.

While it runs, open a **second terminal** and watch the energies appear:

```
cd ~/Downloads/CIS1_Tutorial2_NVE/namd
grep ^ENERGY out/nve.log | tail -3         # latest energies, in kcal/mol
```

The column names are on the line starting with `ETITLE:` (see `4_Cheatsheet_NAMD_vs_OpenMM.pdf`).

Also open `nve.namd` in a text editor and find:

- ☐ `amber`, `parmfile`, `ambercoor`: the input files
- ☐ `cellBasisVector1/2/3`: the box
- ☐ `cutoff 9.0` and `PME yes`: how nonbonded forces are computed
- ☐ `timestep 2.0` and `rigidBonds all`
- ☐ `temperature 300`: the random starting velocities
- ☐ the two `run` lines: 10 ps to settle, then 50 ps that we analyse
- ☐ that there is **no** `langevin` and **no** `langevinPiston` line. This is what makes the run NVE.

NAMD has finished when the last line of the log says `End of program`:

```
tail -3 out/nve.log
```

Step 2. Run OpenMM

In your second terminal (NAMD can keep running in the first):

```
cd ~/Downloads/CIS1_Tutorial2_NVE/openmm
python nve.py 1000                          # 1000 steps x 2 fs = 2 ps, about 2–6 minutes
```

OpenMM on a laptop CPU is slow, so you run only 2 ps. The full 60 ps run is in `reference_results/`.

While it runs, open `nve.py` and match each block to a line in `nve.namd`, using the side-by-side table in the cheat sheet. Find the line that makes this run NVE (`VerletIntegrator`) and check that no barostat is added.

When it finishes, look at the output. Note that OpenMM energies are in **kJ/mol**, not kcal/mol:

```
head -3 nve.csv                             # step, time (ps), PE, KE, total, temperature (K)
```

Step 3. Plot the results

When both runs have finished:

```
cd ../analysis
python plot_nve.py
```

This prints the mean temperature, the fluctuation of the total energy (`std(Etot)`) and its drift for each engine, and saves the plot `nve_energies.png`. Open it:

```
xdg-open nve_energies.png                   # Ubuntu.  On macOS:  open nve_energies.png
```

Your OpenMM run is only 2 ps long, so **ignore its drift number**: it is extrapolated from too little data.

Now plot the full 60 ps reference runs of both engines:

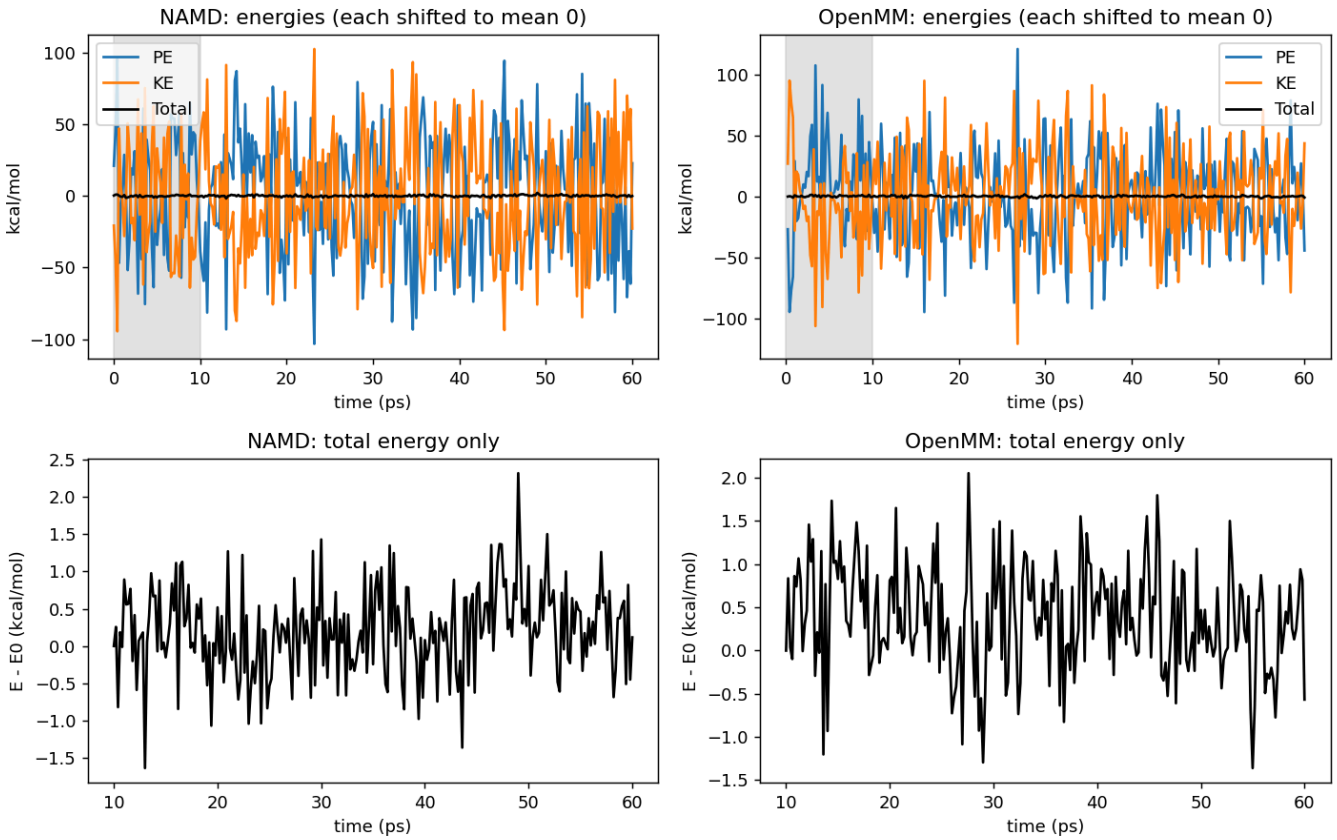
```
python plot_nve.py ../reference_results/namd_nve.log ../reference_results/openmm_nve.csv
```

What you should see

For the 60 ps reference runs (first 10 ps left out):

	mean T	std of E_total	drift
NAMD	297.7 K	0.58 kcal/mol	+6 kcal/mol/ns
OpenMM	298.1 K	0.61 kcal/mol	−6 kcal/mol/ns

KE swings by about ±50 kcal/mol, while E_total stays within about ±1 kcal/mol, out of a KE of about 15,800 kcal/mol. The drift has opposite signs in the two engines and is as small as the noise, so there is **no systematic drift**: energy is conserved. Your NAMD numbers should be close to these.



Step 4. Questions

Write short answers in your notes.

1. In the top plots, which two curves look like mirror images? Why?
2. Which curve stays flat? What law of physics does that show?
3. What is the average temperature? Nothing controls the temperature, so why does it fluctuate? *(Hint: T is proportional to KE.)*
4. What would a thermostat do to the flat curve?

5. The two engines start from different random velocities and use different code. Why do their curves still look statistically the same?
6. Why do we leave out the first 10 ps?

Step 5. Exercise: change the timestep

This is the remaining class time, or homework if NAMD is slow on your laptop.

Predict first: if you make the timestep smaller, will E_total fluctuate more or less? What could go wrong if you make it bigger?

Make a copy of the NAMD file for each new timestep. Keep the simulated time the same (60 ps), so the number of steps must change:

File	`timestep`	first `run`	second `run`	`outputName`
`nve.namd` (done)	2.0	5000	25000	`out/nve`
`nve_dt1.namd`	1.0	10000	50000	`out/nve_dt1`
`nve_dt4.namd`	4.0	2500	12500	`out/nve_dt4`

Also change `outputEnergies` to `200` for 1 fs and to `50` for 4 fs, so that energies are still printed every 0.2 ps.

```
cd ../namd
cp nve.namd nve_dt1.namd                    # then edit the lines in the table above
namd2 +p4 nve_dt1.namd > out/nve_dt1.log
cd ../analysis
python plot_nve.py ../namd/out/nve_dt1.log -      # '-' means: NAMD only, no OpenMM file
```

Compare `std(Etot)` and the drift with your 2 fs run. Was your prediction right? What happened at 4 fs?

Optional: watch the movie in VMD

Ubuntu, from the `namd` folder:

```
vmd ../inputs/trpcage_wat.prmtop -parm7 -dcd out/nve.dcd
```

macOS (or Ubuntu without the command line): in the VMD window, *File → New Molecule*, load `inputs/trpcage_wat.prmtop` as type **parm7**, then load `namd/out/nve.dcd` into the same molecule.

If something goes wrong

Problem	Fix
`EnvironmentNameNotFound: ... cis1-md`	The environment was never created. In the tutorial folder: `conda env create -f environment.yml` then `conda activate cis1-md`
`namd2: command not found`	NAMD is not on your PATH. Find it with `ls -d ~/namd ~/NAMD_*`. Ubuntu: `echo 'export PATH=$HOME/namd:$PATH' >> ~/.bashrc; source ~/.bashrc`. macOS: same with `~/.zshrc`. Replace `namd` with your folder name. Details in the install guide
macOS: *"namd2" cannot be opened because the developer cannot be verified*	`xattr -dr com.apple.quarantine ~/namd` (your NAMD folder)
`out/nve.log: No such file or directory`	Run `mkdir -p out` inside the `namd` folder
`ModuleNotFoundError: No module named 'openmm'`	Run `conda activate cis1-md` first
NAMD stops with `FATAL ERROR`	Read the line after it. If you edited `nve.namd`, copy a fresh one from the zip
`plot_nve.py` says `No ENERGY: lines`	NAMD has not finished or crashed. Check `tail out/nve.log`
OpenMM is very slow	Normal on CPU. Use fewer steps: `python nve.py 500`

Summary

- NVE = Newton's equations only: no thermostat, no barostat.
- Kinetic and potential energy trade off; their sum stays constant, and temperature fluctuates by itself.
- Energy conservation tests the integrator and the settings (timestep, cutoff, PME).
- Real workflow: equilibrate with a thermostat and barostat first, then switch them off for NVE.
- NAMD and OpenMM read the same inputs and give statistically the same answer. They differ only in how you talk to them: a config file versus a Python script.